

Design and Implementation of a Generic Monte Carlo Pricer in Python

Jérôme Lelong

April 13, 2026

The purpose of the project is to design the structure of a tool to price and hedge options using Monte-Carlo methods. Then, the implementation will be carried out in Python.

Section 1 presents the problem and practical motivations. The model used to represent the evolution of asset prices (the underlyings) is described in Section 2. Section 3 asks you to consider the architecture of the pricer.

1 Practical motivations

When studying a derivative product, it is important to be able to compute its price and to construct its hedging portfolio.

Let $S = (S_t, t \geq 0)$ denote the underlying process and $\mathcal{F}$ its natural filtration. In this project we assume that S follows a multidimensional Black Scholes model (see Section 2.2). To keep it simple, we only consider payoffs of the form

$$\varphi(S_{t_0}, S_{t_1}, \dots, S_{t_N})$$

where $0 = t_0 < t_1 < \dots < t_N = T$ is a regular grid of observation dates, i.e. $t_k = kT/N$ for $k = 0, \dots, N$.

Example. Here are some examples of payoff functions φ for a single underlying.

- $N = 1$ and $\varphi(S_0, S_1) = (S_1 - K)_+$ is a vanilla call option.
- $T = 1, N = 12$ and $\varphi(S_0, S_1, \dots, S_{12}) = (S_{12} - K)_+ \mathbf{1}_{\{\forall 0 \leq i \leq 12, S_i \geq L\}}$ is a discretely monitored barrier call (one observation per month). The option is knocked out if the underlying falls below L at any observation date.
- $T = 1, N = 365$ and $\varphi(S_0, \dots, S_{365}) = (\frac{1}{365} \sum_{i=0}^{364} S_i - K)_+$ is an Asian call obtained by replacing the continuous average $\frac{1}{T} \int_0^T S_u du$ by its discrete counterpart.

Let r denote the instantaneous interest rate, which is assumed to be constant in this project. The price at time $0 \leq t \leq T$, with $t_i \leq t < t_{i+1}$, of such an option is given by

$$v(t, S_{t_0}, \dots, S_{t_i}, S_t) = e^{-r(T-t)} \mathbb{E}[\varphi(S_{t_0}, S_{t_1}, \dots, S_{t_N}) | \mathcal{F}_t].$$

This quantity will be computed by Monte Carlo.

Time-0 price. Focusing on the special case of pricing at time 0:

$$v(0, S_0) = e^{-rT} \mathbb{E}[\varphi(S_{t_0}, S_{t_1}, \dots, S_{t_N})]$$

We approximate $v(0, S_0)$ by

$$e^{-rT} \frac{1}{M} \sum_{j=1}^M \varphi(S_{t_0}^{(j)}, S_{t_1}^{(j)}, \dots, S_{t_N}^{(j)})$$

where the N -tuples $(S_{t_0}^{(j)}, S_{t_1}^{(j)}, \dots, S_{t_N}^{(j)})$ for $j = 1, \dots, M$ are i.i.d following the distribution of $(S_{t_0}, S_{t_1}, \dots, S_{t_N})$.

Time- $t > 0$ price. Pricing at time $t > 0$ is slightly more involved because it requires to compute a conditional expectation. When possible, one convenient approach is to write the future underlying S_{t+u} for $u > 0$ as a function of the current value S_t and a quantity independent of $\mathcal{F}_t$. In the Black Scholes model, we can write

$$S_{t+u} = S_t \tilde{S}_u$$

where $\tilde{S}$ has the same law as S but is independent of $\mathcal{F}_t$, i.e. independent of the past up to time t . Thus, the price at time $t > 0$, with $t_{i-1} \leq t_i$ can be written

$$v(t, s_{t_0}, \dots, s_{t_i}, s_t) = e^{-r(T-t)} \mathbb{E}[\varphi(s_{t_0}, \dots, s_{t_i}, s_t \tilde{S}_{t_{i+1}-t}, \dots, s_t \tilde{S}_{t_N-t})].$$

The lower-case letters s denote deterministic values. Then, we can approximate the price at time t by the following Monte Carlo average

$$e^{-r(T-t)} \frac{1}{M} \sum_{j=1}^M \varphi(s_{t_0}, s_{t_1}, \dots, s_{t_i}, s_t \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, s_t \tilde{S}_{t_N-t}^{(j)}) \quad (1)$$

where the vectors $(\tilde{S}_{t_{i+1}-t}^{(j)}, \dots, \tilde{S}_{t_N-t}^{(j)})_j$ are independent of the the past up to time t and i.i.d. with the law of $(\tilde{S}_{t_{i+1}-t}, \dots, \tilde{S}_{t_N-t})$. Particular care must be taken when simulating the vectors $(s_t \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, s_t \tilde{S}_{t_N-t}^{(j)})_j$. The quantities $(s_{t_0}, s_{t_1}, \dots, s_{t_i}, s_t)$ represent observed values up to time t on the trajectory on which the option price is computed. In practice these are the market quotes of the underlying observed up to date t .

2 The Black Scholes Model

2.1 One-dimensional case

Consider a one dimensional Black Scholes model defined by

$$S_t = S_0 e^{(r-\sigma^2/2)t + \sigma B_t},$$

where B is a real valued standard Brownian motion, $r > 0$ the instantaneous interest rate, and $\sigma > 0$ the volatility of the underlying. Note that for all $t, u \geq 0$ we can write

$$S_{t+u} = S_t e^{(r-\sigma^2/2)u + \sigma(B_{t+u} - B_t)} = S_t \tilde{S}_u \quad (2)$$

where $\tilde{S}$ is **independent** of $\mathcal{F}_t$ and the dynamic of $\tilde{S}$ is given by

$$\tilde{S}_u = e^{(r-\sigma^2/2)u + \sigma\tilde{B}_u}$$

where $(\tilde{B}_u = (B_{t+u} - B_t))_{u \geq 0}$ is a real valued standard Brownian motion independent of $\mathcal{F}_t$. Note that $\tilde{S}$ follows the same dynamic as S but with initial value $\tilde{S}_0 = 1$. The discretization of S on the grid $t_0, t_1, \dots, t_N$ is therefore given by

$$S_{t_{i+1}} \stackrel{Law}{=} S_{t_i} e^{(r-\sigma^2/2)(t_{i+1}-t_i) + \sigma\sqrt{t_{i+1}-t_i}G_{i+1}} \quad (3)$$

where the sequence $(G_i)_{i \geq 1}$ is i.i.d. standard normal. Let $t_i \leq t < t_{i+1}$ and we want to sample S conditionally on the information up to time t . It is sufficient to sample $S_{t_{i+1}}$ using s_t (the last known value) and then to apply (3)

$$\begin{cases} S_{t_{i+1}} \stackrel{Law}{=} s_t e^{(r-\sigma^2/2)(t_{i+1}-t) + \sigma\sqrt{t_{i+1}-t}G_{i+1}}, \\ S_{t_{k+1}} \stackrel{Law}{=} S_{t_k} e^{(r-\sigma^2/2)(t_{k+1}-t_k) + \sigma\sqrt{t_{k+1}-t_k}G_{k+1}}, \end{cases} \quad \text{for } i+1 \leq k \leq N-1. \quad (4)$$

2.2 Multidimensional case

Consider D assets each following a one-dimensional Black Scholes model and correlated with each other. The dynamics S of these D assets read

$$S_{t,d} = S_{0,d} e^{(r-(\sigma_d)^2/2)t + \sigma_d B_{t,d}}, \quad d = 1, \dots, D \quad (5)$$

where r is the interest rate, $(\sigma_1, \dots, \sigma_D)$ are the real volatilities of the assets and $B = (B_1, \dots, B_D)^T$ is a vector of D real valued standard Brownian motions with correlation matrix Γ defined by $\Gamma = \frac{\text{Cov}(B_t, B_t)}{t}$. Note that B is not a standard Brownian motion in $\mathbb{R}^D$ since its components are not independent, but we can represent it using a standard $\mathbb{R}^D$ Brownian motion $W = (W^1, \dots, W^D)^T$. We have $(B_t, t \geq 0) \stackrel{Law}{=} (LW_t, t \geq 0)$, where L is the Cholesky factorization of Γ , i.e. $\Gamma = LL'$ where L is a lower-triangular matrix. Then, Equation (5) can be rewritten as

$$S_{t,d} = S_{0,d} e^{(r-(\sigma_d)^2/2)t + \sigma_d L_d W_t}, \quad d = 1, \dots, D$$

where L_d denotes the d^{th} -row of the matrix L . Thus, $L_d W_t$ is a scalar value. From this equation, one easily deduces

$$S_{t_{i+1},d} \stackrel{Law}{=} S_{t_i,d} e^{(r-(\sigma_d)^2/2)(t_{i+1}-t_i) + \sigma_d \sqrt{t_{i+1}-t_i} L_d G_{i+1}}, \quad d = 1, \dots, D$$

where the sequence $(G_i)_{i \geq 1}$ is i.i.d. centered Gaussian vectors with identity covariance in $\mathbb{R}^D$. Thus to simulate S on the grid $(t_i)_{i=0, \dots, N}$ it suffices to simulate i.i.d. standard Gaussian vectors.

We know that, in continuous time, it is possible to construct a perfect hedging portfolio. Thanks to the self-financing assumption, the hedging portfolio is completely determined by the amount of risky assets to hold at each time t , given by the derivative of the price with respect to S_t , i.e. $\frac{\partial v(t, S_{t_0}, \dots, S_{t_i}, S_t)}{\partial S_t}$. These quantities, called the option deltas, are typically approximated using finite differences

$$\begin{aligned} \frac{e^{-r(T-t)}}{M2s_th} \sum_{j=1}^M & \left(\varphi(s_{t_0}, s_{t_1}, \dots, s_{t_i}, s_t(1+h)\tilde{S}_{t_{i+1}-t}^{(j)}, \dots, s_t(1+h)\tilde{S}_{t_N-t}^{(j)}) \right. \\ & \left. - \varphi(s_{t_0}, s_{t_1}, \dots, s_{t_i}, s_t(1-h)\tilde{S}_{t_{i+1}-t}^{(j)}, \dots, s_t(1-h)\tilde{S}_{t_N-t}^{(j)}) \right) \end{aligned} \quad (6)$$

where the sequence $(\tilde{S}_{t_{i+1}-t}^{(j)}, \dots, \tilde{S}_{t_N-t}^{(j)})_j$ is i.i.d. following the law of $(S_{t_{i+1}-t}, \dots, S_{t_N-t})$ and is simulated using (4).

3 Monte Carlo Pricer architecture

3.1 Pricing

The goal of this project is to write a tool that can price any product in a Black Scholes model of arbitrary dimension by specifying only three things: the option payoff, the model parameters and its simulation function `asset`. To that end, you will implement three classes: `MonteCarlo`, `Option`, `BlackScholesModel`.

Question 1. *Propose a global design of the three classes and how they interact.*

Price at time 0 First, we will limit ourselves to computing the price at time 0. Implement the method `price` in the `MonteCarlo` class so that it computes by Monte Carlo the price at time 0 of an option. This function should also compute the width of the 95% confidence interval for the estimator. Recall that the variance of the estimator may be approximated by

$$\xi_M^2 = e^{-2rT} \left[\frac{1}{M} \sum_{j=1}^M (\varphi(S_{t_0}^{(j)}, S_{t_1}^{(j)}, \dots, S_{t_N}^{(j)}))^2 - \left(\frac{1}{M} \sum_{j=1}^M \varphi(S_{t_0}^{(j)}, S_{t_1}^{(j)}, \dots, S_{t_N}^{(j)}) \right)^2 \right]$$

and that a 95% confidence interval is given by

$$\left[e^{-rT} \frac{1}{M} \sum_{j=1}^M \varphi(S_{t_0}^{(j)}, S_{t_1}^{(j)}, \dots, S_{t_N}^{(j)}) \pm \frac{1.96\xi_M}{\sqrt{M}} \right].$$

Question 2. *Write a method `MonteCarlo::price` that calls the `asset` and `payoff` methods.*

Question 3. *Propose some tests for the different features and components of your code.*

Price at a time $t > 0$ Once the price at time 0 is tested and validated, you may proceed to the computation of the price at any time $t > 0$.

To do so, one needs to be able to compute estimator (1). This expectation can be computed by standard Monte Carlo provided one can generate trajectories i.i.d. according to the conditional law of S given $\mathcal{F}_t$, i.e. thanks to property (2) trajectories of the form

$$(s_{t_0}, s_{t_1}, \dots, s_{t_i}, s_t \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, s_t \tilde{S}_{t_N-t}^{(j)}),$$

which can be generated using (4).

Question 4. *Modify the class specifications above to allow simulation of a trajectory conditional on its known past up to a given date.*

Question 5. *Implement the price computation at an arbitrary date.*

Before adding more features, it is important to think about possible code optimizations, in particular the use of workspaces to avoid recreating new options at each Monte-Carlo iteration.

3.2 Computing the delta

We aim at writing a Monte-Carlo method `MonteCarlo::delta` to compute the delta of the option at time $t \geq 0$ using Equation (6). Note that to use this formula, you need to be able to sample two shifted asset paths using the **same Brownian path**. From one sample of the model $(s_{t_0}, s_{t_1}, \dots, s_{t_i}, s_t \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, s_t \tilde{S}_{t_N-t}^{(j)})$, we aim at building the following shifted paths for $d = 1, \dots, D$

$$\begin{aligned} & (s_{t_0}, s_{t_1}, \dots, s_{t_i}, g_{d,+}(s_t) \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, g_{d,+}(s_t) \tilde{S}_{t_N-t}^{(j)}) \\ & (s_{t_0}, s_{t_1}, \dots, s_{t_i}, g_{d,-}(s_t) \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, g_{d,-}(s_t) \tilde{S}_{t_N-t}^{(j)}) \end{aligned}$$

where

$$g_{d,+}(x) = \begin{pmatrix} x_1 \\ \vdots \\ x_{d-1} \\ x_d(1+h) \\ x_{d+1} \\ \vdots \\ x_D \end{pmatrix} \quad g_{d,-}(x) = \begin{pmatrix} x_1 \\ \vdots \\ x_{d-1} \\ x_d(1-h) \\ x_{d+1} \\ \vdots \\ x_D \end{pmatrix}.$$

Then, the deltas $\frac{\partial v(t, s_{t_0}, \dots, s_{t_i}, s_t)}{\partial S_{t,d}}$, for $d = 1, \dots, D$ are approximated by

$$\begin{aligned} & \frac{e^{-r(T-t)}}{M 2 s_{t,d} h} \sum_{j=1}^M \left(\varphi(s_{t_0}, s_{t_1}, \dots, s_{t_i}, g_{d,+}(s_t) \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, g_{d,+}(s_t) \tilde{S}_{t_N-t}^{(j)}) \right. \\ & \quad \left. - \varphi(s_{t_0}, s_{t_1}, \dots, s_{t_i}, g_{d,-}(s_t) \tilde{S}_{t_{i+1}-t}^{(j)}, \dots, g_{d,-}(s_t) \tilde{S}_{t_N-t}^{(j)}) \right). \end{aligned} \quad (7)$$

Question 6. Implement a method `BlackScholesModel::shift_asset`.

Question 7. Implement a method `MonteCarlo::delta`.

4 Hedging portfolio

Among the many criterions to prefer one model over an other, its ability to efficiently hedge exotic options plays a central role. Practitioners compares models in terms of Profit and Loss $P\mathcal{E}L$, representing the hedging error at maturity time.

In this section, we assume to have market data at hand represented by quotes of the model on a regular time grid $0 = \tau_0 < \dots < \tau_H = T$ with step size T/H . We assume that the regular grid of the market quotes $\{\tau_0, \dots, \tau_H\}$ satisfies $\{t_0, \dots, t_N\} \subset \{\tau_0, \dots, \tau_H\}$, see Figure 1.

Let p_0 denote the time-0 price of the price and δ_i the vector of delta at each time τ_i for $i = 1, \dots, H$. Let $S_t^0 = e^{rt}$ be the non-risky asset. The value of the hedging portfolio is given by

$$V_i = \delta_i \cdot S_{\tau_i} + \phi_i^0 S_{\tau_i}^0$$

where ϕ_i^0 is determined using the self-financing equation stating that

$$V_{i+1} - V_i = \delta_i \cdot (S_{\tau_{i+1}} - S_{\tau_i}) + \phi_i^0 (S_{\tau_{i+1}}^0 - S_{\tau_i}^0).$$

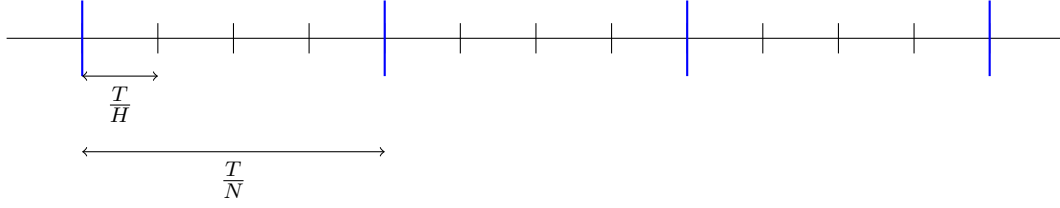


Figure 1: Example with $H = 12$ et $N = 3$.

Let us introduce $\tilde{V}_i = \frac{V_i}{S_{\tau_i}^0}$ and $\tilde{S}_t = \frac{S_t}{S_t^0}$. Then,

$$\tilde{V}_{i+1} - \tilde{V}_i = \delta_i \cdot (\tilde{S}_{\tau_{i+1}} - \tilde{S}_{\tau_i}).$$

The hedging error at maturity time is given by $P\&L = p_0 + e^{r\tau_H} \tilde{V}_H - \text{payoff}$.

Question 8. *Implement a method to compute the hedging portfolio on a given market path. It should compute both the value of the portfolio on the grid $(\tau_i)_{0 \leq i \leq H}$ and the final P&L.*

5 Parameters

The different parameters are specified in a `.json` file of the following type

```
{
  "model type": "bs",
  "option size": 1,
  "strike": 100.0,
  "spot": [
    100.0
  ],
  "maturity": 1.0,
  "volatility": [
    0.2
  ],
  "interest rate": 0.04879,
  "correlation": 0.0,
  "trend": [
    0.04
  ],
  "option type": "basket",
  "payoff coefficients": [
    1.0
  ],
  "fixing dates number": 1,
  "sample number": 50000,
  "hedging dates number": 365,
  "fd step": 0.1
}
```

Key	Type	Name	Key	Type	Name
option size	integer	d	strike	float	K
spot	vector	S_0	option type	string	
maturity	float	T	payoff coefficients	vector	λ
volatility	vector	σ	fixing dates number	integer	N
interest rate	float	r	hedging dates number	integer	H
correlation	float	ρ	fd step	float	h
trend	vector	μ	sample number	integer	M

Some remarks on the parameters

- The following parameters are not used in the project: `model type`, `trend`.
- The correlation parameter ρ is used to specify the correlation matrix Γ : $\Gamma_{ij} = \mathbf{1}_{i=j} + \rho \mathbf{1}_{i \neq j}$ for $i, j = 1, \dots, D$. To ensure that Γ is definite, one must choose $\frac{-1}{D-1} < \rho < 1$.
- If a parameter (for instance S_0, σ, λ) is a vector but a single value is given, the convention is to set all the components of the vector to this value. The actual length of the vector must be determined first.

6 Options to implement

Here is the list of options to be implemented in the project

- Basket option (key `basket`)

$$\left(\sum_{d=1}^D \lambda_d S_{T,d} - K \right)_+$$

where $\lambda_i \in \mathbb{R}$ and $K \in \mathbb{R}$. The case $K \leq 0$ enables us to consider a put like option.

- Discrete Asian option (key `asian`)

$$\left(\sum_{d=1}^D \lambda_d \frac{1}{N+1} \sum_{i=0}^N S_{t_i,d} - K \right)_+.$$

- Performance option on a basket (key `performance`)

$$1 + \sum_{i=1}^N \left(\frac{\sum_{d=1}^D \lambda_d S_{t_i,d}}{\sum_{d=1}^D \lambda_d S_{t_{i-1},d}} - 1 \right)_+.$$

7 Expected output

Your code will run as

```
python ./mcpricer.py market.txt params.json outfile.json
```

where

- `market.txt` is a file containing the market quotes. It can be read using `numpy.loadtxt`.

- `params.json` is a file describing the option as the one presented in Section 5
- `outfile.json` is created by your program and should contain on exit an array of size H with every entry being a dictionary with keys

```
{
  "date":
  "value":
  "price":
  "priceStdDev":
  "deltas":
  "deltasStdDev":
}
```

where

- `date` is the current date as an integer $i \in \{0, \dots, H\}$;
- `value` is V_{τ_i} ;
- `price` is the price at time τ_i ;
- `priceStdDev` is the standard deviation of the Monte-Carlo est estimator of the price at time τ_i ;
- `deltas` is the vector of deltas at time τ_i ;
- `deltasStdDev` is is the vector of standard deviations of the Monte-Carlo estimator of each deltas at time τ_i .

This output will be obtained by creating a `Portfolio` object and calling `.toJson()`.

8 Design suggestion

